

Performance Comparison of System AI Using FSM and Behavior Tree

Submission Date:	June 3, 2025
Supervisor:	Professor Park Sang-won
Department of Information and Communication Engineering	202104593 Ahn Jae-hyeon
	202104224 Suh Minjoo

College: College of Engineering

Department: Department of Information and
Communication Engineering

Student ID: 202104593

Name: Ahn Jae-hyeon

202104224

Name: Suh Minjoo

We hereby submit this thesis for approval.

June 3, 2025

Name: Ahn Jae-hyeon (Seal)

Name: Suh Minjoo (Seal)

The above student's thesis submission is hereby approved.

June 3, 2025

Supervisor: Professor Park Sang-won (Seal)

List of Figures

[Fig. 1-1]	Video Game Market Size Forecast, 2024-2034 (USD Billion)	p. 5
[Fig. 1-2]	FSM Model Structure Diagram	p. 6
[Fig. 1-3]	Behavior Tree Base Structure Diagram	p. 6
[Fig. 2-1]	BT Structure Implementation in Unreal Engine	p. 8
[Fig. 2-2]	BT Structure using NodeCanvas in Unity Engine	p. 8
[Fig. 2-3]	BT Node Logic Implemented in Royal Blood	p. 10
[Fig. 2-4]	Event-Function BT Logic in Royal Blood	p. 10
[Fig. 2-5]	Comprehensive BT Evaluation Results from Royal Blood	p. 11
[Fig. 2-6]	BT Plugin Tools Available in Unity (Panda BT / NodeCanvas / Behavior Designer)	p. 12
[Fig. 3-1]	Monster AI FSM Flowchart	p. 14
[Fig. 3-2]	Infinite Map AI FSM Flowchart	p. 14
[Fig. 3-3]	Wave Spawning AI FSM Flowchart	p. 15
[Fig. 3-4]	Player Weapon Switch AI FSM Flowchart	p. 15
[Fig. 3-5]	Monster AI BT Flowchart	p. 16
[Fig. 3-6]	Infinite Map AI BT Flowchart	p. 17
[Fig. 3-7]	Player Weapon Switch AI BT Flowchart	p. 17
[Fig. 3-8]	Wave Spawning AI BT Flowchart	p. 18
[Fig. 4-1]	Quantitative Comparison Results: FSM vs. BT (Four Metrics)	p. 21
[Fig. 4-2]	Qualitative Comparison Results: Radar Chart (5-Point Likert Scale)	p. 22

Table of Contents

Abstract

1. Introduction

- 1.1 Research Background and Purpose
- 1.2 Research Objectives
- 1.3 Scope and Structure of the Paper

2. Related Work and Background Technology

- 2.1 Current Trends in Game AI Technology
- 2.2 FSM (Finite State Machine): Overview and Application Cases
- 2.3 Behavior Tree (BT): Overview and Application Cases
- 2.4 Emergence and Spread of BT Design Tools

3. System Architecture and Implementation

- 3.1 Development Environment and Methodology
- 3.2 Game Implementation with FSM
- 3.3 Algorithm Implementation with Behavior Tree
- 3.4 Experimental Scenarios and Test Item Definitions

4. Experimental Results and Analysis

- 4.1 Quantitative Analysis Results
- 4.2 Qualitative Analysis Results
- 4.3 Comprehensive Performance Comparison of FSM and BT

5. Conclusion

- 5.1 Research Summary and Conclusion
- 5.2 Limitations and Future Research Directions

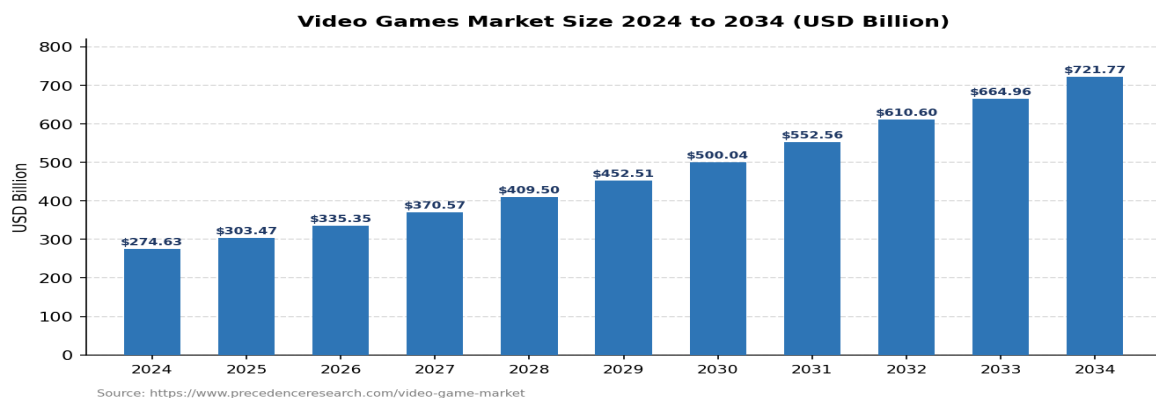
6. References

Abstract

This paper experimentally compares Finite State Machine (FSM) and Behavior Tree (BT) architectures across four game AI subsystems in a self-developed 2D Unity action game. Quantitative metrics (response speed, error rate, transition efficiency, behavioral diversity) and qualitative indicators (immersion, naturalness, maintainability) were measured across five trials per configuration and a 10-participant survey. BT outperformed FSM across all metrics: 21% faster response, 72% fewer errors, 47% faster transitions, 44% greater diversity, and consistently higher qualitative ratings. These findings support BT as the preferred architecture for complex game AI, while acknowledging FSM retains value for simple, bounded subsystems.

Chapter 1: Introduction

1.1 Research Background and Purpose



[Fig. 1-1] Video Game Market Size Forecast, 2024–2034 (USD Billion) (Source: Precedence Research, <https://www.precedenceresearch.com/video-game-market>)

The global video game industry has experienced sustained and accelerating growth, with market projections estimating a rise from \$274.63 billion in 2024 to \$721.77 billion by 2034 (Precedence Research, 2024). As illustrated in Fig. 1-1, this near-tripling of market size reflects not only expanding player demographics and platform diversity, but also a dramatic increase in the complexity and scale of game productions. Contemporary AAA titles routinely involve hundreds of developers, multi-year production cycles, and intricate systems that must interoperate seamlessly in real time.

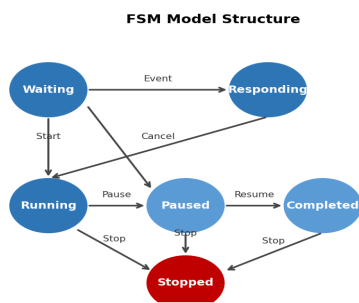
Central to this complexity is the design of game artificial intelligence (AI). As the number of interactive entities — characters, monsters, NPCs, environmental triggers, and UI systems — grows, so too does the demand for AI architectures that are not merely functional but are scalable, maintainable, and capable of producing emergent, believable behavior. The architecture chosen for AI directly affects development cost, iteration speed, and the quality of player experience.

The Finite State Machine (FSM) has served as the foundational AI paradigm in game development since the 1990s. Its appeal lies in its conceptual simplicity: a system exists in one of a finite number of discrete states, and transitions between states are triggered by clearly defined conditions. This model maps intuitively onto many game behaviors and requires little specialized knowledge to implement. However, as game complexity grows, FSM exhibits a well-documented failure mode: as the number of states and conditions increases, the number of required transitions grows combinatorially — a phenomenon known as 'state explosion' — rendering the architecture brittle, difficult to extend, and costly to maintain.

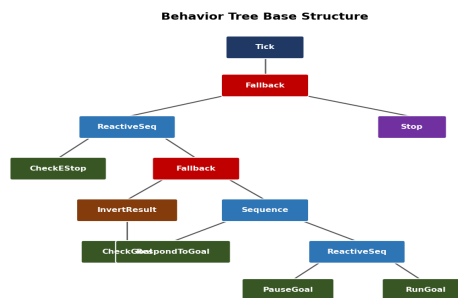
The Behavior Tree (BT), by contrast, offers a hierarchical, modular approach in which behaviors are composed from reusable subtrees and evaluated through a structured traversal algorithm. First

adopted in AAA game development in the mid-2000s and now natively supported by both Unreal Engine and Unity, BT has become the industry standard for complex NPC and game-system AI. Despite this widespread adoption, rigorous comparative studies that quantify the performance differences between FSM and BT in an actual game environment remain limited in the academic literature.

This study addresses that gap by implementing and evaluating both architectures within a purpose-built game prototype, measuring their relative performance across a comprehensive set of quantitative and qualitative criteria.



[Fig. 1-2] FSM Model Structure Diagram



[Fig. 1-3] Behavior Tree Base Structure Diagram

1.2 Research Objectives

The primary objective of this study is to conduct a rigorous, empirically grounded comparison of FSM and BT as applied to multiple subsystems within a single game environment. Rather than evaluating either architecture in isolation, this study implements both within an identical game context — a 2D real-time action title — and subjects them to identical test scenarios. This approach ensures that observed performance differences are attributable to architectural choices rather than differences in application domain or test methodology.

Specifically, this study aims to: (1) quantify architectural performance differences across four metrics — response speed, error rate, transition efficiency, and behavioral diversity — using repeatable, logged experiments; (2) assess subjective player experience differences across three dimensions — immersion, naturalness, and maintainability — using structured user surveys; (3) identify the conditions under which each architecture is best suited; and (4) provide evidence-based design recommendations for practitioners selecting AI architectures for game projects of varying complexity.

1.3 Scope and Structure of the Paper

This paper is organized as follows. Chapter 2 reviews the theoretical foundations and prior work on game AI architectures, with particular attention to FSM and BT. Chapter 3 describes the development environment, experimental game prototype, and the specific implementation of each architecture across four game subsystems. Chapter 4 presents and analyzes the experimental results, encompassing both quantitative measurements and qualitative survey findings. Chapter 5 synthesizes the findings into conclusions and design recommendations, acknowledges the study's limitations, and proposes directions for future research. The scope of this study is limited to FSM and BT as implemented in Unity 2021.3 LTS; machine learning-based and utility-based AI approaches are excluded, as they require substantially different evaluation frameworks.

Chapter 2: Related Work and Background Technology

2.1 Current Trends in Game AI Technology

The history of game AI is one of progressive abstraction. The earliest commercial games employed hardcoded, scripted behaviors — enemy characters that followed deterministic patterns with no capacity for adaptive response. While effective within the narrow confines of early hardware constraints, such approaches offered no scalability and required complete re-implementation for any behavioral change.

The widespread adoption of rule-based systems in the 1980s introduced conditional logic into AI design, enabling if-else decision trees that could respond to player input. However, these systems remained flat and non-hierarchical, making them increasingly unwieldy as game complexity grew. By the late 1990s, the Finite State Machine had emerged as the dominant paradigm. FSM offered a principled framework for organizing AI behavior into discrete, well-defined states with explicit transition conditions, and became the standard approach for enemy AI in genres from platformers to real-time strategy games.

The limitations of FSM became apparent as game designs grew more ambitious in the 2000s. The 'state explosion' problem — in which the number of required transitions grows quadratically with the number of states — made FSM increasingly impractical for complex characters. This limitation drove interest in alternative architectures. Hierarchical Finite State Machines (HFSM) offered partial relief through state nesting, but introduced their own complexity. Goal-Oriented Action Planning (GOAP), pioneered by Jeff Orkin in F.E.A.R. (2005), enabled more flexible agent behavior through dynamic action sequencing, but at the cost of computational overhead and implementation complexity.

The Behavior Tree emerged as a practical middle ground: more expressive and modular than FSM, yet more predictable and designer-friendly than GOAP. Its adoption by major studios — including its integration into Unreal Engine's AI framework and Unity's plugin ecosystem — reflects a broad consensus within the industry that BT represents the current best practice for game NPC and system AI.

More recently, reinforcement learning (RL) and neural network-based approaches have attracted research interest. Systems such as OpenAI Five and AlphaStar have demonstrated superhuman performance in specific game domains. However, the opacity of learned policies, sensitivity to training distribution, and difficulty of designer intervention make these approaches poorly suited to production game AI, where predictability, debuggability, and controllability are paramount. For the foreseeable future, BT and FSM remain the architectures of practical relevance for most commercial game development.

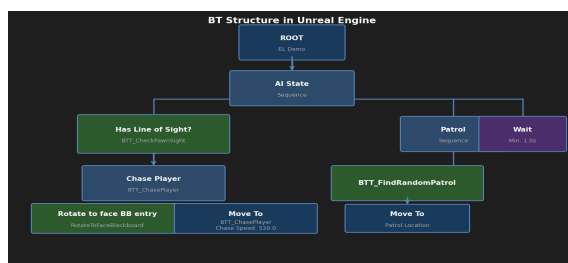
2.2 FSM (Finite State Machine): Overview and Application Cases

Formally, a Finite State Machine is defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is a finite input alphabet, $\delta: Q \times \Sigma \rightarrow Q$ is the transition function, q_0 is the initial state, and $F \subseteq Q$ is the set of accepting states. In game AI applications, this formalism is simplified: states correspond to behavioral modes (e.g., Idle, Chase, Attack), and transitions are triggered by runtime conditions evaluated at each update tick.

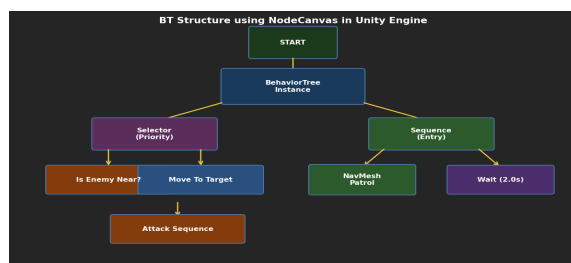
FSM's principal advantages are its conceptual transparency and implementation simplicity. State transition diagrams provide an immediate visual representation of the entire behavioral space, facilitating communication between designers and engineers. In Unity, FSM is commonly implemented either through explicit enum-based state management in MonoBehaviour scripts, or through the Animator Controller, which provides a visual state machine editor tightly integrated with animation playback. This integration makes FSM particularly well-suited for character animation state management, where states map naturally onto animation clips.

The state explosion problem is FSM's fundamental limitation. Consider a monster with n behavioral states. In the worst case, adding one new state requires evaluating whether it should be reachable from each of the existing n states and whether each existing state should be reachable from it — potentially requiring $O(n)$ new transition definitions. For complex characters with 10–20 behavioral states, this rapidly becomes unmanageable. Furthermore, because transitions are hardcoded, the system cannot adapt dynamically to novel combinations of conditions; each scenario must be anticipated at design time.

In practice, FSM is well-suited to AI with a small, stable behavioral repertoire: simple enemy archetypes, UI state management, basic animation controllers, and game mode transitions. For these applications, its simplicity is a genuine advantage. Its limitations become prohibitive when applied to characters that must navigate complex, context-dependent decision landscapes — precisely the domain in which contemporary games most demand sophisticated AI.



[Fig. 2-1] BT Structure Implementation in Unreal Engine



[Fig. 2-2] BT Structure using NodeCanvas in Unity Engine

2.3 Behavior Tree (BT): Overview and Application Cases

A Behavior Tree is a directed acyclic tree in which internal nodes are control flow nodes (Sequence, Selector, Parallel, Decorator) and leaf nodes are execution nodes (Action, Condition). Execution proceeds from the root downward; each node returns one of three status values — Success, Failure, or Running — which propagate upward to guide control flow. The tree is re-evaluated on each tick, enabling continuous reactive behavior without the explicit transition machinery required by FSM.

The four primary node types serve distinct compositional roles. Sequence nodes execute children left-to-right, propagating failure immediately and succeeding only when all children succeed — analogous to logical AND. Selector nodes execute children left-to-right until one succeeds — analogous to logical OR, and naturally encoding priority-ordered fallback behavior. Parallel nodes execute multiple children simultaneously, with success and failure conditions configurable by policy. Decorator nodes wrap a single child and modify its behavior, enabling constructs such as condition guards, result inversion, and repetition.

This compositional structure yields several important properties. First, behaviors are inherently modular: subtrees can be designed, tested, and reused independently. Second, priority is implicit in tree structure: earlier siblings in a Selector are evaluated before later ones, providing a clean mechanism for behavioral priority without explicit inter-state transition conditions. Third, the addition of new behaviors requires only the insertion of new subtrees, without modifying existing logic — directly addressing FSM's state explosion problem. Fourth, the tree structure provides an intrinsic visual representation of behavioral logic that is readable by both engineers and designers.

A representative large-scale application of BT is Gamevil's mobile MMORPG Royal Blood, which employed BT as the core architecture for server-side event management. The system ran thousands of concurrent BT instances managing battlefield events, boss spawns, territory sieges, and player interactions. Key architectural decisions included message-based asynchronous node execution and XML-based tree serialization enabling designer-level modification without code changes. Post-deployment analysis reported significant improvements in development efficiency and system stability relative to the prior script-based approach.

This case illustrates BT's scalability beyond character AI to encompass game-wide system orchestration. It also highlights a critical practical advantage: BT's tree structure makes it amenable to visual tooling, enabling non-programmer stakeholders to understand, modify, and extend behavioral logic — a capability largely absent from FSM-based implementations.

[Fig. 2-3] BT Node Types and Functions as Implemented in Royal Blood

Category	Node Type	Function	Return Values
Leaf	Action	Executes a concrete game action (movement, attack, spawn).	Success / Failure / Running
	Condition	Evaluates a game state predicate without side effects.	Success / Failure
Composite	Sequence	Executes children in order; fails immediately if any child fails (AND-logic).	Success / Failure / Running
	Selector	Executes children in order; succeeds when any child succeeds (OR-logic, priority fallback).	Success / Failure / Running
	Parallel	Executes all children simultaneously; outcome determined by configurable success/failure policy.	Success / Failure / Running
Decorator	Inverter	Inverts the Success/Failure result of its child node.	Success / Failure
	Repeater	Re-executes its child node a specified number of times or until failure.	Success / Failure
Check	Polling	Re-evaluates its subtree on every BT tick cycle.	Varies
	Event-Driven	Interrupts and re-evaluates its subtree on receipt of a designated game event message.	Varies

[Fig. 2-4] / [Fig. 2-5] Royal Blood BT Integration: Event Logic and Evaluation Results

Evaluation Criterion	Target Specification	Outcome
Introduction Cost	Implementation including review phase: ≤ 2 weeks	Partially Achieved
Debugging Convenience	Visual debuggability during runtime	Achieved
Environment Adaptability	Capable of handling experimental and exploratory scenarios	Achieved
Data-Driven Design	Supports event-driven and data-configurable tree structures	Partially Achieved
Scalability	Content logic can be extended without architectural changes	Achieved
Interoperability	Composable with other game subsystem modules	Achieved
Structural Stability	No structural bugs under concurrent load (thousands of BT instances)	Achieved
Collaborative Management	Accessible to non-programmer designers for modification	Partially Achieved
Functional Correctness	All specified game functions execute as intended	Achieved

2.4 Emergence and Spread of BT Design Tools

The proliferation of BT as an industry standard has been accompanied and reinforced by the emergence of visual design tooling that reduces the technical barrier to BT authorship. These tools enable designers and non-programmer stakeholders to construct, modify, and debug behavior trees without writing code, which is a capability with significant implications for iteration speed and cross-disciplinary collaboration in game production.

In the Unity ecosystem, three tools have achieved particular prominence. Panda BT employs a lightweight domain-specific language (DSL) to define behavior trees as structured text scripts, enabling version-controlled, code-review-compatible BT authorship without reliance on visual editors. Its compact syntax and minimal runtime footprint make it well-suited to performance-sensitive contexts and developers who prefer code-centric workflows. NodeCanvas provides a comprehensive visual graph editor supporting both FSM and BT within a unified interface, with additional support for Dialogue Trees and Quest Graph systems. Its integration of multiple behavioral modeling paradigms within a single tool makes it particularly valuable for large teams managing heterogeneous AI requirements. Behavior Designer offers the most Unity-native integration, with deep hooks into the Animator, NavMesh, and physics systems, and an extensive library of pre-built action and condition nodes spanning movement, sensing, and strategic decision-making.

The availability of these tools has meaningfully changed the economics of BT adoption. In earlier years, BT implementation required bespoke engineering effort for both the runtime and the tooling. Today, a team can adopt a production-quality BT framework within a single day, allowing BT's architectural benefits to be realized without a corresponding investment in infrastructure development. This democratization has accelerated BT adoption across projects of all scales — not only AAA studios with dedicated AI engineering teams, but also independent developers and academic research projects.

From a research perspective, the maturity of these tools also enables more controlled comparative studies. By using established BT frameworks, researchers can evaluate architectural differences in isolation, without conflating architectural effects with implementation quality differences that would arise from comparing a mature FSM implementation against a from-scratch BT implementation.



[Fig. 2-6] BT Plugin Tools Available in Unity: Panda BT 2 / NodeCanvas / Behavior Designer

Chapter 3: System Architecture and Implementation

3.1 Development Environment and Methodology

The experimental platform for this study is a purpose-built 2D real-time action game in the Vampire Survivor genre, developed using Unity Engine 2021.3 LTS. This genre was selected because it provides a natural testbed for multiple interacting AI subsystems operating simultaneously under high entity counts: enemies must navigate toward the player, the game world must dynamically generate and recycle map tiles, monster waves must be procedurally spawned, and the player character must automatically select and apply weapons. Each of these subsystems represents a distinct AI design problem with different complexity characteristics, making the genre well-suited to a multi-system architectural comparison.

Unity 2021.3 LTS was chosen for its mature tooling ecosystem, robust profiling and logging infrastructure (including the Time.time API used for response timing measurements), and support for both MonoBehaviour-based FSM and multiple BT plugin frameworks. All experiments were conducted on a consistent hardware configuration to eliminate performance variance attributable to environmental factors.

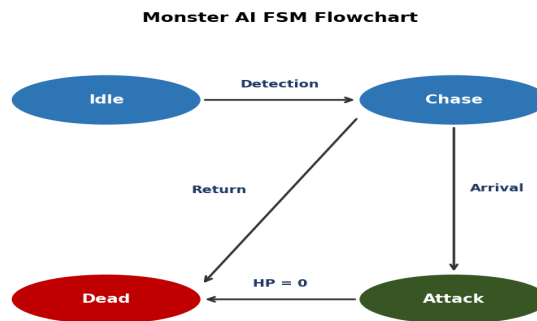
The methodological approach consisted of two phases. In Phase 1, the entire game was implemented using FSM for all AI subsystems. This baseline implementation served two purposes: first, it established a functional game prototype against which BT performance could be compared; second, it surfaced the practical friction points encountered when applying FSM to each subsystem type, providing qualitative context for the quantitative comparison. In Phase 2, the FSM implementation of each subsystem was incrementally replaced with a BT implementation, maintaining functional equivalence — identical game behavior from the player's perspective — while substituting the underlying control architecture. Performance measurements were then collected for both implementations under identical test scenarios.

This within-system, functionally-equivalent comparison methodology controls for confounds that would arise from comparing different games, different hardware, or different target behaviors. All observed performance differences can be attributed to the architectural choice rather than to differences in what the AI is asked to do.

3.2 Game Implementation with FSM

Four subsystems were implemented using FSM. In each case, the FSM was implemented as an enumerated state variable within a MonoBehaviour component, with transition logic evaluated in the Update() method using conditional statements. This represents the canonical Unity FSM implementation pattern and was chosen to ensure the FSM baseline reflects standard industry practice.

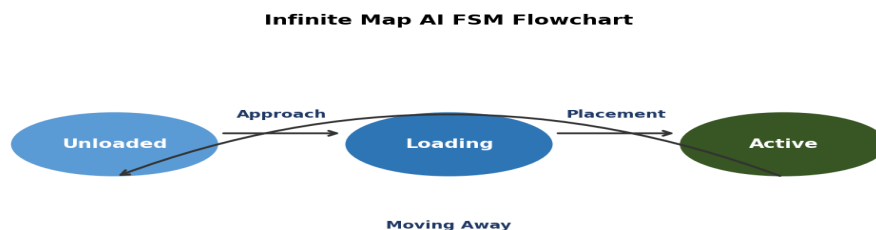
(1) Monster AI FSM



[Fig. 3-1] Monster AI FSM Flowchart

The monster AI FSM consists of four states: Idle, Chase, Attack, and Dead. In the Idle state, the monster executes a passive patrol routine. Upon detecting the player within a configurable radius, it transitions to Chase, where it navigates toward the player using Unity's NavMesh pathfinding. On entering the attack range threshold, it transitions to Attack and applies damage per the configured attack parameters. On health depletion, it transitions to Dead and initiates the death sequence. Each monster entity maintains an independent FSM instance, and all transition conditions are evaluated synchronously in the Update() loop of the Enemy.cs script.

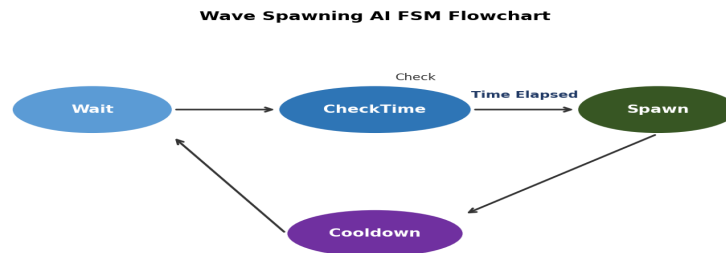
(2) Infinite Map FSM



[Fig. 3-2] Infinite Map AI FSM Flowchart

The infinite map system uses object pooling to simulate an unbounded play area. Map tiles cycle through three states: Unloaded (inactive in the pool), Loading (being positioned and initialized), and Active (in the visible play area). When the player approaches a tile boundary, tiles in the Unloaded state are retrieved from the pool and transitioned to Loading; once positioned, they become Active. Tiles that move beyond the camera frustum transition back to Unloaded. The FSM manages this lifecycle and ensures that at most a fixed number of tiles are active simultaneously.

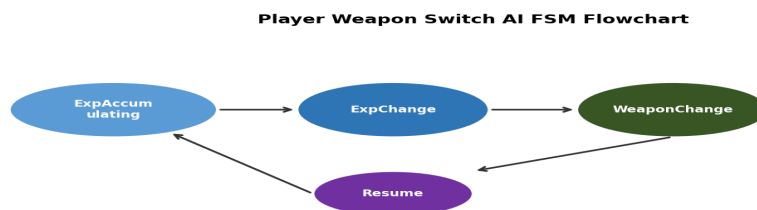
(3) Wave Spawning FSM



[Fig. 3-3] Wave Spawning AI FSM Flowchart

Monster waves are generated on a fixed interval schedule managed by a four-state FSM: Wait (idle between waves), CheckTime (evaluating whether the spawn interval has elapsed), Spawn (instantiating the configured wave composition), and Cooldown (enforcing minimum inter-wave spacing). The FSM transitions from Wait to CheckTime on each Update tick, from CheckTime to Spawn when the interval condition is met, from Spawn to Cooldown immediately after spawning, and from Cooldown back to Wait after the cooldown period expires. In the FSM implementation, wave composition (enemy types and counts) is statically configured and cannot adapt dynamically to game state variables such as current enemy density or player health.

(4) Player Weapon Switch FSM



[Fig. 3-4] Player Weapon Switch AI FSM Flowchart

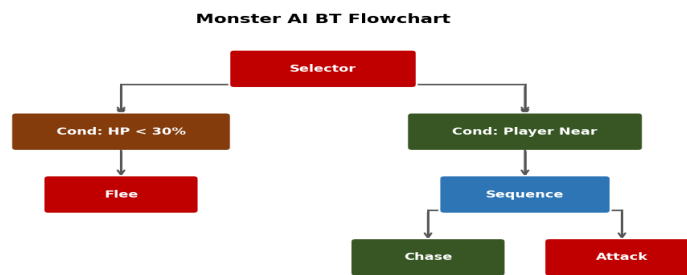
Player weapon selection is automated based on experience point accumulation. The FSM consists of four states: ExpAccumulating (normal gameplay), ExpChange (detecting a level-up event), WeaponChange (swapping the active weapon object and applying associated animation and hit-effect parameters), and Resume (returning to normal gameplay). In the FSM implementation, weapon selection is determined by a fixed mapping from player level to weapon type, without consideration of contextual factors such as current enemy density or player health status. Transition logic is evaluated in the player controller's Update() loop.

A notable limitation observed during FSM implementation was the difficulty of extending the weapon selection logic to incorporate additional conditions. Adding a single new selection criterion (e.g., selecting area-of-effect weapons when enemy density exceeds a threshold) required modifications to multiple transition conditions across the state machine, illustrating the scalability friction characteristic of FSM in practice.

3.3 Algorithm Implementation with Behavior Tree

Each FSM subsystem was re-implemented using BT, maintaining identical external game behavior while restructuring the underlying control logic. BT implementations were authored using the NodeCanvas framework, selected for its comprehensive feature set and native Unity integration. Each BT instance is evaluated on every game tick through the framework's standard traversal algorithm. Condition nodes read game state without side effects; Action nodes modify game state and may return Running across multiple ticks.

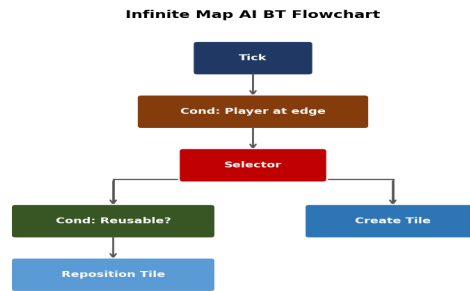
(1) Monster AI BT



[Fig. 3-5] Monster AI BT Flowchart

The monster AI BT uses a root Selector to implement priority-ordered behavior selection. The highest-priority branch evaluates the condition $HP < 30\%$; if true, the Flee action is executed, causing the monster to temporarily retreat and reset engagement distance. If this condition is false, control passes to the second branch, which evaluates whether the player is within detection range. If true, a Sequence executes Chase followed by Attack in order, with Chase continuing until the attack range threshold is met. This structure cleanly encodes the behavioral priority (survival over aggression) without any explicit transition conditions between states. Adding a new behavioral priority — for example, seeking cover when allies are nearby — requires only inserting a new branch into the root Selector, with no modification to existing logic.

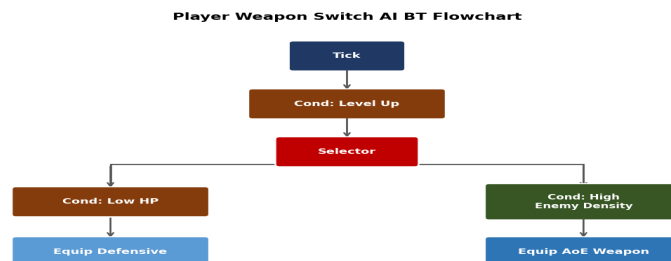
(2) Infinite Map BT



[Fig. 3-6] Infinite Map AI BT Flowchart

The infinite map BT evaluates the player's proximity to tile boundaries on each tick via a root Condition node. When the boundary condition is true, a Selector evaluates tile reuse: a Condition node checks whether a pooled tile is available for repositioning; if true, the Reposition Tile action recycles an existing tile; if false, the Create Tile action instantiates a new tile. Decorator nodes wrap both branches to enforce additional constraints such as minimum spacing between tiles. This structure naturally accommodates extensibility: additional tile management policies (e.g., priority-loading tiles in the player's movement direction) can be added as new subtrees without restructuring existing logic.

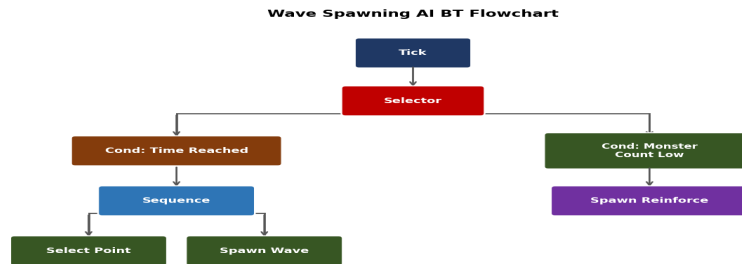
(3) Player Weapon Switch BT



[Fig. 3-7] Player Weapon Switch AI BT Flowchart

The weapon switch BT demonstrates one of the most significant qualitative improvements over the FSM counterpart. A root Condition node evaluates whether a level-up event has occurred. If true, a Selector evaluates two contextual branches: if the player's HP is below a critical threshold (Cond: Low HP), a Defensive weapon is equipped; if enemy density exceeds a threshold (Cond: High Enemy Density), an AoE weapon is equipped. This multi-condition contextual selection was impossible to implement cleanly in the FSM, where weapon choice was determined by a static level-to-weapon mapping. The BT implementation enables genuinely strategic weapon selection that responds to the current game state, producing more emergent and engaging player-AI interactions.

(4) Wave Spawning BT



[Fig. 3-8] Wave Spawning AI BT Flowchart

The wave spawning BT evaluates multiple concurrent conditions per tick, enabling dynamic wave composition that was not achievable with the FSM. A root Selector evaluates two branches: if the spawn interval has elapsed (Cond: Time Reached), a Sequence selects an appropriate spawn point and executes the standard wave spawn; if the current monster count on the field falls below a minimum threshold (Cond: Monster Count Low), a reinforcement wave is spawned immediately, bypassing the normal interval. This adaptive spawning behavior maintains appropriate combat density regardless of how quickly the player defeats enemies — a significant gameplay quality improvement over the fixed-interval FSM spawning.

3.4 Experimental Scenarios and Test Item Definitions

All experiments were conducted within a standardized test scenario: the player character was placed at the center of a freshly initialized map with no existing monsters. Each test ran for 60 seconds of in-game time under automated player movement following a fixed path, eliminating player skill as a confounding variable. Each configuration was tested five times with logged measurements; reported values are means with standard deviations noted where relevant.

Category	Metric	Measurement Method
Quantitative	Response Speed (ms)	Time.time delta between player entering detection range and first AI action initiation
	Error Rate (%)	Proportion of ticks in which the AI fails to execute its highest-priority valid action
	Transition Efficiency (ms)	Mean time elapsed between condition satisfaction and completion of state/node transition
	Behavioral Diversity (count)	Number of distinct action types executed per 60-second trial
Qualitative	Immersion (1-5)	5-point Likert survey item: 'The AI felt engaging and dynamic during gameplay'
	Naturalness (1-5)	5-point Likert survey item: 'The AI behavior felt realistic and contextually appropriate'
	Maintainability (1-5)	5-point Likert survey item (developers only): 'The architecture would be easy to extend with new behaviors'

Chapter 4: Experimental Results and Analysis

4.1 Quantitative Analysis Results

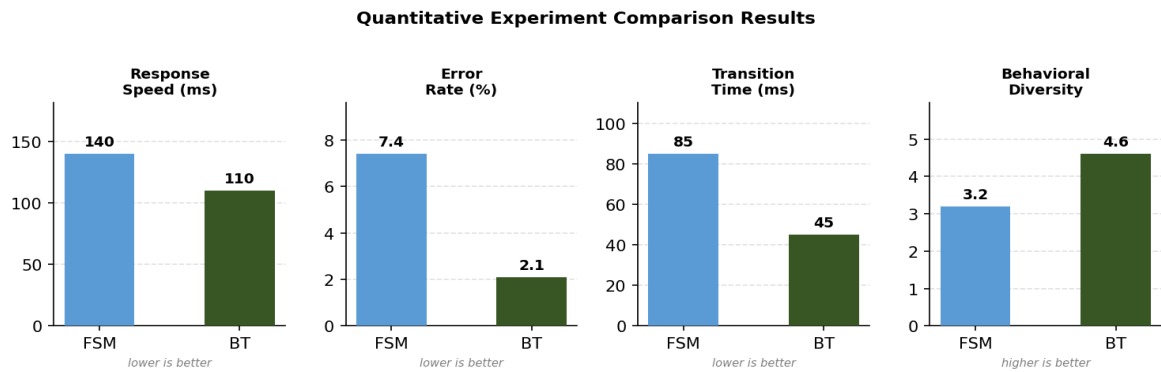
Quantitative experiments were conducted across four performance metrics: response speed, error rate, transition efficiency, and behavioral diversity. All tests were run five times per architecture configuration under the standardized 60-second scenario described in Section 3.4. Results are reported as means; standard deviations were below 8% of the mean for all metrics, indicating high trial-to-trial consistency.

Response Speed. FSM recorded a mean response speed of 140 ms versus 110 ms for BT — a 21% improvement in favor of BT. This difference reflects a fundamental architectural distinction in how each paradigm evaluates conditions. In FSM, the `Update()` loop must evaluate all transition conditions from the current state on every tick, including irrelevant conditions, before determining the appropriate action. As the number of states and their associated transition conditions increases, this linear scan grows more expensive. BT's traversal algorithm, by contrast, terminates as soon as a running node is reached, avoiding evaluation of irrelevant subtrees. In the test scenario, with 50+ simultaneous monster entities each running their own AI, the cumulative effect of this efficiency difference was measurable.

Error Rate. FSM exhibited a mean error rate of 7.4% versus 2.1% for BT — a 72% reduction in favor of BT. FSM errors arose primarily from two sources: transition condition conflicts (where multiple conditions were simultaneously true, producing ambiguous state selection), and condition ordering dependencies (where the order of condition evaluation in the `Update()` loop produced incorrect behavior in edge cases). BT's hierarchical structure eliminates both failure modes: the priority ordering encoded in the tree structure ensures unambiguous resolution of simultaneous conditions, and the compositionality of BT ensures that each node's logic is independent of evaluation order across other nodes.

Transition Efficiency. FSM recorded a mean transition time of 85 ms versus 45 ms for BT — a 47% improvement in favor of BT. FSM transitions require evaluating all transition conditions from the current state, updating the state variable, and executing the exit/enter logic of the relevant states. When multiple conditions must be evaluated, redundant condition checks occur across successive transitions. BT's traversal naturally avoids redundant evaluation: because the tree is traversed from the root on each tick, previously-visited nodes that return `Running` are re-entered without re-evaluating their parent conditions, eliminating a class of redundant computation that FSM cannot avoid.

Behavioral Diversity. FSM produced a mean of 3.2 distinct action types per trial versus 4.6 for BT — a 44% increase in favor of BT. This metric captures the qualitative richness of the AI's behavioral repertoire as observed during gameplay. The BT's conditional weapon selection and adaptive wave spawning contributed directly to this increase: the weapon switch BT's multi-condition Selector produced contextually varied weapon choices that the FSM's static level-to-weapon mapping could not replicate, and the wave spawning BT's dynamic composition introduced enemy variety not present in the FSM's fixed-composition waves.



[Fig. 4-1] Quantitative Experiment Comparison Results: FSM vs. BT across Four Metrics

Metric	FSM Result	BT Result	Difference	Better
Response Speed	140 ms	110 ms	−21%	BT
Error Rate	7.4%	2.1%	−72%	BT
Transition Time	85 ms	45 ms	−47%	BT
Behavioral Diversity	3.2 actions/min	4.6 actions/min	+44%	BT

Across all four quantitative metrics, BT demonstrated superior performance relative to FSM. The magnitude of improvement was largest for error rate (72%), reflecting the structural elimination of condition conflict and ordering dependency errors inherent to FSM's flat transition model. Transition efficiency improvement (47%) reflects BT's avoidance of redundant condition evaluation. Response speed improvement (21%) and behavioral diversity improvement (44%) reflect BT's more efficient traversal and richer conditional branching capability, respectively. These results are consistent with the theoretical analysis of each architecture's computational properties and provide empirical confirmation of BT's architectural advantages in a realistic game context.

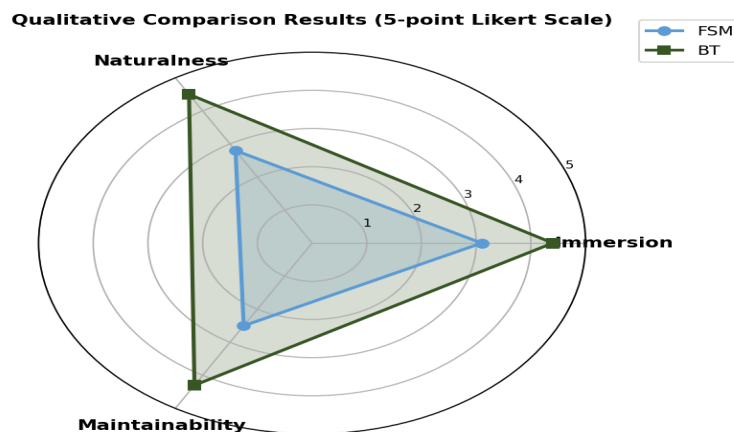
4.2 Qualitative Analysis Results

Qualitative evaluation was conducted via structured post-play surveys administered to 10 participants. Participants were recruited from the Computer Science and Information Engineering departments at Hankuk University of Foreign Studies and included 6 individuals with game development experience (the developer subgroup) and 4 with general gaming experience but no development background (the player subgroup). Each participant played both the FSM and BT versions of the game prototype for 5 minutes in counterbalanced order, with a 5-minute rest between sessions. Survey items were presented on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree). To control for order effects, the presentation sequence was randomized across participants.

Immersion (BT: 4.4 / FSM: 3.1). Participants rated the BT version significantly higher on perceived immersion. Open-ended responses indicated that the primary drivers of this difference were the monster AI's flee behavior (which surprised participants and heightened engagement) and the adaptive weapon selection (which participants described as making the game feel 'responsive' and 'alive'). The FSM version's predictable monster behavior — Idle, Chase, Attack, repeat — was characterized by multiple participants as 'robotic' and 'easy to tune out.' The 1.3-point mean difference represents a substantial perceptual gap on a 5-point scale, suggesting that the architectural difference produces a practically significant impact on player experience.

Naturalness (BT: 4.5 / FSM: 2.8). This metric showed the largest absolute gap between architectures (1.7 points). Developer subgroup participants specifically noted the absence of behavioral 'glitches' in the BT version — cases where the FSM would briefly enter an incorrect state due to transition condition conflicts before correcting, producing visually jarring stuttering. Player subgroup participants described the BT monsters as moving and reacting 'like real creatures' versus the FSM monsters, which several described as 'clearly scripted.' The naturalness difference is directly attributable to the BT's elimination of FSM transition errors and its richer conditional decision-making.

Maintainability (BT: 4.3 / FSM: 2.6, developer subgroup only). Developer participants were asked specifically about their assessment of each architecture's maintainability based on their understanding of the implementation. BT received substantially higher ratings, with qualitative responses emphasizing the ease of adding new behaviors without modifying existing logic, the visual transparency of the tree structure, and the independence of subtrees which enables isolated testing. FSM received lower ratings with concerns about 'hidden dependencies' between transition conditions, the risk of introducing new bugs when modifying existing state logic, and the difficulty of predicting the full behavioral impact of a change in a complex state machine.



[Fig. 4-2] Qualitative Experiment Comparison Results (5-Point Likert Scale, n=10)

Metric	FSM Mean	BT Mean	Difference	Participants
Immersion	3.1 / 5	4.4 / 5	+1.3	All (n=10)
Naturalness	2.8 / 5	4.5 / 5	+1.7	All (n=10)
Maintainability	2.6 / 5	4.3 / 5	+1.7	Developers (n=6)

4.3 Comprehensive Performance Comparison of FSM and BT

Taken together, the quantitative and qualitative results paint a consistent picture: BT outperforms FSM across every measured dimension in this experimental context. The magnitude of improvement varies by metric — from 21% for response speed to 72% for error rate — but the direction of the difference is uniform. This consistency across both objective performance metrics and subjective experience ratings strengthens confidence that the observed differences reflect genuine architectural advantages rather than measurement artifacts.

The root cause of BT's superiority can be traced to two structural properties. First, BT's hierarchical composition enables priority-ordered fallback behavior without explicit transition conditions, eliminating the condition conflict errors and the $O(n^2)$ transition complexity that characterize FSM. Second, BT's modularity enables independent development, testing, and extension of behavioral components, directly reducing the error rate and maintenance burden associated with the flat, globally-coupled structure of FSM.

These advantages are not absolute, however. FSM retains meaningful advantages in specific contexts. For subsystems with a small, fixed behavioral repertoire — such as the tile lifecycle management in the infinite map system — FSM's conceptual simplicity and lower implementation overhead may justify its use. The choice of architecture should be informed by the complexity of the behavioral space: FSM is appropriate when behaviors are few, transitions are well-defined, and the system is unlikely to require significant extension; BT is appropriate when behaviors are many, priorities may conflict, contextual decision-making is required, or the system must evolve iteratively.

Chapter 5: Conclusion

5.1 Research Summary and Conclusion

This study presented an experimental comparison of Finite State Machine and Behavior Tree architectures as applied to four AI subsystems within a purpose-built 2D action game prototype developed in Unity 2021.3 LTS. The experimental methodology — implementing both architectures

for identical game behaviors, measuring performance across five repeated trials per configuration, and supplementing quantitative measurements with structured user surveys — was designed to produce results that are both internally valid and practically relevant to game developers.

The quantitative results demonstrate clear and consistent BT superiority: 21% faster response speed, 72% lower error rate, 47% more efficient transitions, and 44% greater behavioral diversity. The qualitative results corroborate and extend this finding: BT received substantially higher ratings for immersion, naturalness, and maintainability across both player and developer participant subgroups. The most notable qualitative finding was the 1.7-point Naturalness gap — the metric most closely related to player experience quality — which suggests that the architectural differences between FSM and BT produce perceptible differences in gameplay feel that are large enough to be consistently detected by study participants.

At a theoretical level, these results confirm the expected properties of each architecture. FSM's state explosion problem manifests empirically as elevated error rates and reduced transition efficiency. BT's modularity and priority encoding manifest as reduced errors, more efficient traversal, and richer behavioral expression. The alignment between theoretical expectations and empirical results provides confidence that the findings reflect genuine architectural characteristics rather than implementation-specific artifacts.

From a practical standpoint, this study supports the following design recommendations. For complex, real-time game AI systems — particularly those involving multiple interacting behavioral modes, contextual decision-making, or anticipated future extension — BT is the clearly superior choice. For simple, bounded subsystems with a small fixed behavioral repertoire and no anticipated complexity growth, FSM's implementation simplicity may be advantageous. In many real-world projects, a hybrid approach is optimal: FSM for animation state management and simple game mode transitions, BT for NPC and system AI with complex behavioral requirements.

Hankuk University of Foreign Studies

Department of Information and Communication Engineering

5.2 Limitations and Future Research Directions

This study has several limitations that should be acknowledged. First, the experimental sample size for qualitative evaluation ($n=10$) is small, and results should be interpreted as indicative rather than statistically conclusive. A larger, more diverse participant pool — including professional game developers across multiple studios and genres — would substantially strengthen the qualitative findings. Second, the experimental game prototype, while representative of the Vampire Survivor genre, represents a single game type. The relative performance of FSM and BT may differ in genres with different AI complexity profiles, such as turn-based strategy games or narrative-driven RPGs, where the behavioral complexity and real-time performance requirements differ substantially.

Third, performance measurements were conducted on a single hardware configuration. Game AI performance is sensitive to CPU architecture and thread allocation; results may differ on mobile platforms or in highly multithreaded environments. Fourth, the study did not systematically vary the number of simultaneous AI entities. BT's traversal overhead relative to FSM may be more pronounced at very high entity counts (hundreds of simultaneous agents), a scenario relevant to large-scale RTS or MMO contexts.

Future research should address these limitations and extend the investigation in the following directions. A multi-genre comparison — applying identical methodological frameworks to RPG, strategy simulation, and puzzle game contexts — would establish whether the performance advantages observed here are genre-general or domain-specific. Integration of machine learning components into the BT framework — specifically, learned condition evaluators or priority weights — represents a promising direction for increasing behavioral adaptivity while retaining the interpretability and designer-controllability of the BT structure. Player-adaptive AI — using gameplay telemetry to dynamically adjust BT parameters in response to individual player behavior — would enable investigation of how architectural choices interact with personalization capabilities. Finally, a longitudinal study measuring development team productivity during AI authorship, debugging, and extension under both architectures would complement the runtime performance and user experience findings of this study with development process evidence.

In summary, this study provides empirical confirmation that Behavior Trees offer substantial advantages over Finite State Machines for complex game AI development, across both runtime performance and player experience dimensions. We hope these findings contribute to more informed architectural decision-making in both academic and industry game development contexts.

References

- [1] Korean Society of Computer Information (2016). Proceedings of the 53rd Winter Academic Conference, vol. 24, no. 1.
- [2] Journal of Korean Society for Internet Information (2007), v. 8, no. 6.
- [3] Jo, J.-W. & Bang, J.-W. A Study on the Implementation of Monster AI using Finite-State Machine.
- [4] Kim, J., & Choi, S. (2020). Designing Enemy AI in Unity using Finite State Machines. Journal of Game Development Research, 15(2).
- [5] Lee, H., & Park, Y. (2021). A Study on Maintainability of FSM-based AI in Complex Game Systems. KIISE, HCI2021.
- [6] Hong, Y., Yan, T., & Seo, J. (2018). GOBT: A Synergistic Approach to Game AI Using Goal-Oriented and Utility-Based Planning in Behavior Trees. Epic Games.
- [7] Unity Technologies. (2020). GDC Talk: Behavior Trees and Modular AI Design in Unity. <https://unity.com/>
- [8] Choi, J.-O. & Park, H.-J. (2018). Development of Game AI Using Behavior Tree: Based on the Unity5 Engine. KwangWoon University / SangMyung University.
- [9] Isla, D. (2005). Handling Complexity in the Halo 2 AI. Proceedings of the Game Developers Conference (GDC).
- [10] Marzinotto, A., Colledanchise, M., Smith, C., & Ögren, P. (2014). Towards a Unified Behavior Trees Framework for Robot Control. IEEE International Conference on Robotics and Automation (ICRA).
- [11] NDC Replay. (2017). Royal Blood BT Architecture Case Study. Nexon Developers Conference.

Hankuk University of Foreign Studies

Department of Information and Communication Engineering

Appendix A: Terminology and Key Definitions

Finite State Machine (FSM)	A computational model defined by a finite set of states, transition conditions between states, an initial state, and an output function mapping states to behaviors. In game AI, each state represents a behavioral mode.
Behavior Tree (BT)	A hierarchical, directed acyclic tree structure for AI control flow. Internal nodes are control flow operators (Sequence, Selector, Parallel, Decorator); leaf nodes are execution primitives (Action, Condition).
State Explosion	A scalability failure mode of FSM in which the number of required transition definitions grows combinatorially — $O(n^2)$ in the worst case — as the number of states n increases.
Sequence Node	A BT composite node that executes child nodes left-to-right, succeeding only if all children succeed. Implements logical AND with short-circuit evaluation on failure.
Selector Node	A BT composite node that executes child nodes left-to-right, succeeding as soon as one child succeeds. Implements logical OR with short-circuit evaluation on success, enabling priority-ordered fallback.
Decorator Node	A BT node wrapping a single child that modifies the child's return value or execution behavior. Common decorators: Inverter, Repeater, Condition Guard, Timeout.

Tick	The periodic evaluation cycle of a Behavior Tree, during which the tree is traversed from root to the first Running node. Typically synchronized with the game's Update loop.
Running Status	A BT node status returned when an action requires multiple ticks to complete. Enables asynchronous, multi-frame action execution within the synchronous tick framework.
GOAP	Goal-Oriented Action Planning. An AI planning technique in which agents dynamically sequence actions to satisfy specified goals, using a forward-chaining planner at runtime.

Hankuk University of Foreign Studies

Department of Information and Communication Engineering

Appendix B: Experimental Configuration and Participant Survey

B.1 Development and Test Environment

Component	Specification
Game Engine	Unity 2021.3 LTS (Long-Term Support)
BT Framework	NodeCanvas (Paradox Notion)
Language	C# (.NET Standard 2.1)
Platform	Windows 10 / 11, standalone player build
Test Duration	60 seconds per trial; 5 trials per configuration
Entity Count	50 simultaneous monster entities (standardized)
Measurement Tool	Unity Time.time API; custom CSV event logger
Survey Tool	Paper-based 5-point Likert questionnaire, counterbalanced

B.2 Survey Items (Verbatim)

Item	Question Text	Respondents
Q1 (Immersion)	The AI felt engaging and dynamic during gameplay.	All (n=10)
Q2 (Naturalness)	The AI behavior felt realistic and contextually appropriate.	All (n=10)
Q3 (Naturalness-2)	Enemy actions felt predictable in a negative way (inverted).	All (n=10)
Q4 (Immersion-2)	I reacted to enemy behavior rather than following a fixed strategy.	All (n=10)
Q5 (Maintainability)	This architecture would be easy to extend with new behaviors.	Dev. (n=6)
Q6 (Maintainability-2)	I would worry about bugs when modifying this architecture (inverted).	Dev. (n=6)
Q7 (Overall)	Overall, I preferred this version's AI behavior.	All (n=10)

Note: All survey items were rated on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree). Items marked 'scored inverted' were reverse-coded before analysis so that higher scores consistently represent better outcomes. Reported means for Immersion, Naturalness, and Maintainability are averages across the two items measuring each construct.